



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering – Department of Electrical Engineering

**CpE 418: SOFTWARE DESIGN
FINAL PROJECT**

**DEPLOYMENT, DEMONSTRATION, AND DESIGN
EVALUATION (Phase 5)**

Name: Carpio, Patrick B.
Catacutan, Maximus Yajzel
Pasahol, Benedict Stephen
Samson, Dirk P.
Villaflor, James V

Date: May 24, 2026

Section: CpE - 3202

Score: _____

Objective:

Implement and test a functional software system based on approved designs.

Final Requirement Tasks

1. System Deployment

- Ensure that all core features are functional.
- Prepare basic setup or execution instructions.

2. System Demonstration

Students must conduct a live or recorded system demonstration that includes:

- Overview of the system purpose and scope
- Execution of key functionalities
- User interaction flow
- Demonstration of at least one realistic use case

The demonstration must clearly show that the system is functional, stable, and aligned with the approved design.

3. Design Evaluation and Justification

Students must explain and justify their software design decisions, including:

- Selected system architecture and programming paradigm
- Major system modules and their responsibilities



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering – Department of Electrical Engineering

- Functional correctness
- Key data structures or data models
- Consideration of non-functional requirements
- Design trade-offs and constraints encountered

4. Reflection and Future Improvements

Students must provide a reflection discussing:

- Challenges encountered during development
- Lessons learned throughout the project
- Limitations of the current system
- Proposed future improvements or extensions

Final Deliverables

1. Functional and running software system
2. Complete source code (repository or ZIP file)
3. Live or recorded system demonstration (5-10 minutes)
4. Final project report summarizing design decisions and evaluation

Assessment Focus

- System functionality and correctness
- Consistency between design and implementation
- Quality of design justification
- Professional quality of demonstration and documentation
- Depth of reflection and evaluation

Important Notes

- Only projects that have successfully completed Phases 1-4 are eligible for Phase 5.
- Non-functional or incomplete systems may receive grade penalties.
- Academic honesty and proper citation of external resources are required.

Prepared by:

Engr. MARK JOHN FEL T. RAYOS

Course Instructor

Engr. JOYCE ANN A. CUETO

Course Instructor



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering – Department of Electrical Engineering

I. System Overview

A. Purpose and Scope

Traditional coastal surveys are time-consuming and cover limited ground. SEASCAN takes freely available Sentinel-2 satellite imagery (European Space Agency, 2015), runs it through a trained 1D-CNN model built on PyTorch (PyTorch contributors, 2019), and returns classified results in one session. Researchers and coastal managers can then store those results in a SQLite database (SQLite consortium, 2000), compare them over time, and track how coastal ecosystems shift across seasons or years. The primary users include environmental monitoring agencies, academic research teams, and local government units responsible for coastal resource management.

B. Key Features

- **User Authentication**
 - Users log in through a secure login screen. The backend checks credentials against auth.db, issues a JWT token, and routes all subsequent data operations to that user's private database at user_dbs/<username>.db. Admin accounts can create additional users through the API.
- **Satellite Image Upload and Classification**
 - From the Dashboard, the user selects or drags in a GeoTIFF file or a ZIP archive of Sentinel-2 band images. The file goes to the POST /predict endpoint, where the backend extracts geospatial metadata (CRS, bounding box), reads the 12 Sentinel-2 spectral bands and four derived indices (NDVI, NDWI, MNDWI, Brightness), normalizes the feature vector, and runs it through the Coastal1DCNN model. The system returns a classified image (base64 PNG) with a pixel count breakdown per class and a confidence score.
- **Saving a Classification**
 - After reviewing the result, the user enters a study area name and optional notes, then clicks Save. The frontend calls POST /save-classification with all 24 normalized metadata fields, and the backend writes the record to the user's SQLite database while creating or updating the corresponding study_areas entry.
- **Trend Analytics**
 - On the Analytics page, the user selects a study area from the dropdown. The frontend pulls all historical classifications for that area via GET /classifications and plots water, seagrass, sand, and cloud pixel counts on a Recharts line chart across dates. Total classifications, date range, and average confidence appear as summary statistics alongside the chart.
- **Geospatial Map View**
 - The Map page loads a Leaflet interactive map centered on the saved study area's bounding box, with the classified image overlay positioned at the correct geographic coordinates.
- **PDF Report Export**
 - Users export a trend report via POST /export-analytics-pdf. The report contains a time-series trend chart with a class color legend (teal for water, green for seagrass, yellow for sand, gray for clouds), classification



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering – Department of Electrical Engineering

thumbnails, and date-ordered summary data. Generation uses Matplotlib and ReportLab.

C. Realistic Use Case Walkthrough

The following scenario walks through a complete end-to-end workflow under realistic conditions:

1. A coastal researcher downloads the latest Sentinel-2 imagery over Batangas Bay from the Copernicus Open Access Hub (free, no registration required).
2. They log into SEASCAN and upload the TIFF file via the Dashboard.
3. The system classifies the image and reports that 42% is open water, 31% is seagrass meadow, 19% is sandy substrate, and 8% is cloud cover, with a confidence score of 0.87.
4. The researcher saves the result under "Batangas Bay" with a note about the season.
5. One year later, the same workflow is repeated with a new image. The Analytics page shows a trend line where seagrass coverage has dropped by 8% and sand coverage has increased, which may point to substrate exposure from erosion or runoff.
6. The researcher exports a PDF report and submits it to the local government unit to support a coastal rehabilitation proposal.

II. Design Evaluation and Justification

A. System Architecture and Programming Paradigm

SEASCAN uses a three-tier client-server architecture: a React frontend (Meta Open Source, 2013) for presentation, a FastAPI backend (Ramírez & FastAPI contributors, 2019) for application and business logic, and SQLite databases (SQLite consortium, 2000) alongside model weights for data storage. Three considerations shaped this structure.

First, separation of concerns. ML inference, data persistence, authentication, and geospatial logic each live in separate modules. A change to one layer does not require touching the others, which keeps the codebase testable and maintainable.

Second, REST over HTTP. The JSON-based API decouples the frontend from the backend, so any future replacement for the React frontend (such as a mobile app) can reuse existing endpoints without modification.

Third, SQLite was selected over a server-based RDBMS for its simplicity and portability. For the data volumes expected in this academic project, it is sufficient, and it removes the setup overhead that a full database server would require.

B. Programming Paradigm

The backend uses an object-oriented paradigm for the ML model (Coastal1DCNN as a PyTorch nn.Module subclass) and a functional/procedural style for the API request handlers and database functions. Python type hints and Pydantic validation models enforce correctness at the API boundary. The frontend uses React's functional component paradigm with hooks (useState, useEffect, useMemo) for state management. TypeScript is used throughout the frontend to eliminate implicit any types and catch interface mismatches at compile time.

C. Major System Modules



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering – Department of Electrical Engineering

The backend has four modules, each with a defined scope:

a. **classifier_ui.py** (640+ lines)

- The central FastAPI application. Exposes 12 REST endpoints covering authentication, file upload, ML inference, classification persistence, study area queries, and PDF export. Orchestrates the entire classification pipeline by calling `geospatial.py` for metadata, the `Coastal1DCNN` model for inference, and `database.py` for persistence.

b. **database.py** (450+ lines)

- The SQLite ORM and persistence layer. Manages two databases: a centralized `auth.db` for user accounts and per-user databases in `user_dbs/<username>.db` for classification data. Implements CRUD operations for classifications and study areas, and includes `_ensure_table_columns()` for backward-compatible schema migration.

c. **geospatial.py** (60+ lines)

- Extracts CRS and bounding box metadata from GeoTIFF files using Rasterio. Called by the `/predict` endpoint to enrich classification results with spatial context. The extracted bounds are stored in the database and used to position overlays on the Leaflet map.

d. **Coastal1DCNN** (model)

- A 1D Convolutional Neural Network implemented in PyTorch. Takes a 16-element feature vector per pixel (12 Sentinel-2 spectral bands plus NDVI, NDWI, MNDWI, and Brightness indices) and outputs a 4-class probability distribution. Architecture: three `Conv1d-BatchNorm-ReLU-MaxPool` blocks followed by `Dropout(0.4)` and two linear layers with Softmax output. Trained for 500 epochs with Adam ($lr=5e-4$) on the `Coastal_Area_Data` ground-truth dataset.

e. **React Frontend Components**

- Eleven modular components handle distinct UI responsibilities: `DashboardPage` for upload and classification, `AnalyticsPage` and `TrendAnalysisChart` for trend visualization, `MapPage` for geospatial rendering, `LocationSelector` for study area filtering, and `api.ts` as the centralized fetch layer. This component decomposition follows the single-responsibility principle and allows individual components to be tested or replaced independently.

D. Data Models

The classifications table stores 24 fields per record: pixel counts per class, a confidence score, bounding box as JSON, the CRS code, original filename, and a base64-encoded PNG of the classified overlay. The `study_areas` table aggregates metadata per location (classification count, first and last dates), which avoids repeated joins on classification records when pulling trend data.

Field naming is flat and consistent across all modules. Names like `study_area_location` and `study_area_bounds` appear the same in the API, database, and frontend. That wasn't always the case early on. An inconsistency in field names between layers caused a round of integration bugs that took longer than expected to trace. Standardizing fixed it.

E. Non-Functional Requirements



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering – Department of Electrical Engineering

a. Performance:

- Model inference on 16-feature vectors is near real-time. ONNX export is supported for further inference speedup. Database queries use indexed `study_area_name`. The `TrendAnalysisChart` uses React `useMemo` to avoid unnecessary re-renders.

b. Security:

- JWT-based authentication with configurable secrets via environment variables. Per-user databases ensure data isolation. Pydantic models validate all API inputs before processing.

c. Maintainability:

- TypeScript enforces type safety on the frontend. Python type hints and Pydantic models enforce correctness on the backend. Module separation (API, DB, ML, Geospatial) limits the blast radius of any change.

d. Testability:

- All database functions, model behavior, and geospatial utilities are independently testable. The test suite (`test_suite.py`) achieves 100% pass rate across 15 test cases.

e. Portability:

- SQLite requires no server installation. The frontend build produces a static bundle (775 KB) deployable to any static host. Docker deployment is recommended for production but not required for development.

F. Trade-offs

a. SQLite vs. PostgreSQL:

- SQLite was chosen for simplicity and zero-configuration setup appropriate for the academic scope. The trade-off is limited concurrency under high write loads. A production system serving multiple simultaneous analysts would benefit from PostgreSQL.

b. 1D-CNN vs. spatial CNN:

- A 1D-CNN operating on spectral feature vectors was chosen over a 2D spatial CNN because Sentinel-2 spectral signatures are highly discriminative for the four target classes, and per-pixel classification is computationally efficient. The trade-off is loss of spatial context (e.g., texture features), which a 2D CNN would capture.

c. Base64 image storage:

- Storing classified images as base64 BLOBs in SQLite simplifies the architecture by avoiding file system management, but increases database size significantly for large images. The trade-off is acceptable for the current data volumes.



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering – Department of Electrical Engineering

III. Project Evaluation

A. Challenges Encountered During Development

One of the most significant challenges encountered during development was schema normalization mid-project. The field originally named `location_description` had to be renamed to `study_area_location` after the initial schema was already deployed, which required propagating the change across `classifier_ui.py`, `database.py`, the frontend TypeScript interfaces, and the test suite simultaneously. This also necessitated the implementation of the `_ensure_table_columns()` migration utility to handle backward compatibility without wiping existing data.

A second challenge was cross-platform encoding. The test suite initially used Unicode checkmark characters in its printed output, which caused encoding errors when run on Windows with non-UTF-8 console codepages. This required replacing Unicode symbols with plain ASCII equivalents, a subtle but disruptive issue that appeared only in specific environments.

Handling the variability of GeoTIFF files also proved difficult. Sentinel-2 imagery products come packaged in different formats: single-band and multi-band TIFFs, ZIP archives, and files with inconsistent or missing metadata fields. Making `extract_geospatial_metadata()` robust enough to handle all these cases required careful defensive coding with try-except blocks and sensible fallback values throughout.

Maintaining the frontend-to-backend JSON contract was another persistent source of friction. TypeScript interfaces in `types.ts` had to exactly mirror the Python Pydantic models in `classifier_ui.py`, and any mismatch, such as bounds being represented as a Python dict versus a TypeScript object, produced subtle runtime errors that were difficult to trace without a formal API specification document.

B. Lessons Learned

The most important lesson learned was the value of finalizing the data model before writing code. The schema normalization issue that consumed significant development time could have been entirely prevented with a more thorough design phase. The cost of renaming a field after deployment, including database migration, multi-file updates, and test suite corrections, far exceeded what a careful upfront design would have required.

Working with TypeScript and Python type hints also reinforced how much time a strong type system saves during debugging. Interface mismatches that would otherwise have surfaced as obscure runtime errors were caught at compile time, and the discipline of maintaining typed interfaces across both languages proved to be consistently worthwhile. Similarly, the importance of test isolation became clear when database tests began interfering with each other due to shared state. Tests must operate on isolated, temporary databases to produce reliable and repeatable results regardless of execution order.

The decision to keep `geospatial.py` as a standalone module separate from `classifier_ui.py` demonstrated in practice how modular design pays dividends: the geospatial functions could be tested independently without importing the entire FastAPI application, and their behavior could be understood in isolation. Lastly, the incomplete docstring and JSDoc coverage made code review and onboarding noticeably slower toward the end of the project. Documentation should be treated as a first-class deliverable and written alongside the code, not deferred until after features are complete.



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering – Department of Electrical Engineering

C. Limitations of the Current System

SEASCAN currently supports only a single trained model with no mechanism for versioning or ensemble predictions. If a newer, more accurate model is trained, it cannot coexist with the current model in the system, and there is no way to compare outputs across model versions for the same image. Related to this, error recovery is limited: the frontend retry helper handles transient HTTP failures, but the backend returns an immediate error for corrupted or unsupported TIFF files with no guided recovery path for the user.

On the security side, the JWT authentication implementation does not support token refresh, password rotation, account lockout after repeated failed login attempts, or multi-factor authentication. These are standard requirements for any system handling sensitive environmental data in a production context. The system also lacks a functioning automated CI/CD pipeline; the GitHub Actions workflow was added but has not completed a confirmed successful run, meaning regression testing continues to rely on manual execution.

From a scalability standpoint, SQLite's write lock serializes all concurrent writes, which becomes a bottleneck when multiple users classify images simultaneously. Additionally, storing classified images as base64 BLOBs directly in the SQLite database causes the database file to grow rapidly, making this approach unsustainable for large imagery datasets over time.

D. Proposed Future Improvements

The most impactful near-term improvement would be migrating from SQLite to PostgreSQL. PostgreSQL supports concurrent writes, row-level security, and full-text search, and its use of Alembic for migration management would replace the current manual `_ensure_table_columns()` approach with a versioned, auditable migration history. Alongside this, classified image storage should be moved from base64 BLOBs in the database to a file system or object storage service such as MinIO or AWS S3, with only the file path stored in the database record.

Containerizing the application with Docker and a `docker-compose.yml` file would resolve environment setup inconsistencies and enable one-command deployment, making the system accessible to non-developer users such as field researchers and government agency staff. Authentication should also be hardened with JWT refresh tokens, password reset via email, account lockout policies, and optional two-factor authentication using TOTP.

On the machine learning side, implementing a model registry would allow multiple trained model versions to coexist and enable ensemble voting across versions to improve accuracy on ambiguous spectral signatures. Expanding the classification vocabulary to include coral reefs, mangroves, and built-up coastal structures would broaden the system's practical utility for coastal resource agencies. Finally, refactoring the React frontend to be fully responsive for tablet and mobile devices would allow field researchers to access trend data and review classification results directly from the coast during active surveys, closing the gap between the monitoring system and the real-world decisions it is meant to support.



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2121 / 2221

E-mail Address: coe.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Engineering – Department of Electrical Engineering

IV. SEASCAN Zip File

https://drive.google.com/file/d/1mwr7PbMX0mdLuJz0rdHjUeecqMm7iUHK/view?usp=drive_link

V. References

- Ramírez, S., & FastAPI contributors. (2019). *FastAPI: Modern, fast web framework for building APIs with Python*. Retrieved from <https://fastapi.tiangolo.com>
- PyTorch contributors. (2019). *PyTorch: An imperative style, high-performance deep learning library*. Retrieved from <https://pytorch.org>
- Meta Open Source. (2013). *React: A JavaScript library for building user interfaces*. Retrieved from <https://react.dev>
- Vite contributors. (2020). *Vite: Next generation frontend tooling*. Retrieved from <https://vitejs.dev>
- Gillies, S., & Rasterio contributors. (2013). *Rasterio: Access to geospatial raster data*. Retrieved from <https://rasterio.readthedocs.io>
- The Leaflet contributors. (2010). *Leaflet: An open-source JavaScript library for mobile-friendly interactive maps*. Retrieved from <https://leafletjs.com>
- Recharts Group. (2015). *Recharts: A composable charting library built on React components*. Retrieved from <https://recharts.org>
- ReportLab Inc. (1999). *ReportLab: PDF generation library for Python*. Retrieved from <https://www.reportlab.com>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- European Space Agency. (2015). *Sentinel-2: ESA's optical high-resolution mission for GMES operational services*. Retrieved from <https://sentinel.esa.int/web/sentinel/missions/sentinel-2>
- SQLite consortium. (2000). *SQLite: A self-contained, serverless, zero-configuration SQL database engine*. Retrieved from <https://www.sqlite.org>